Graph Neural Networks that Generalise to Unseen Domains and Boundary Conditions

Research gap survey, version 1: arXiv sources

Yapi Donatien Achou
Computational Science: Physics, University of Oslo
Prepared with AI assistance for literature retrieval and drafting

17 August 2026

1 Thesis objective

Develop a graph neural network that learns physical laws from simulation data, and determine whether it generalises to unseen boundary and initial conditions and unseen domains.

This is studied at two scales:

Continuum mechanics. Porous media flow: Darcy flow, transport, poroelasticity.

Quantum mechanics. Kohn-Sham density functional theory, the self-consistent field problem.

At each scale the model is used in two ways:

Surrogate mode. The model replaces the solver, and is evaluated on its accuracy for conditions and geometries it has not seen.

Warm-start mode. The model gives the solver a good first guess to start from. Solvers work by improving a guess repeatedly until the answer is accurate enough, so a better starting guess means fewer rounds of work. The solver still decides when it is finished, so correctness is guaranteed by the solver rather than by the model, and the model is evaluated only on the iterations and wall-clock time it saves.

What counts as unseen at the quantum scale. The external potential plays the role that boundary conditions and coefficients play at the continuum scale. It is set by the nuclei, which is chemistry, and by the treatment of the cell, meaning an isolated molecule against a periodic crystal. So unseen boundary conditions and geometry becomes unseen chemistry and unseen cell type.

Unseen chemistry and unseen cell type are both stated as open in the literature surveyed below: chemistry by the six open questions on foundation-model potentials [28], and transfer to periodic systems by the limitations section of the solver-aligned initialisation work [31].

One precision. One trained model per scale, not one model spanning both. What transfers across scales is the architecture family, the question being asked, and the evaluation protocol, not the weights.

2 What this document is

This is a survey of the methods that apply machine learning to differential equations, written to find out what is already solved and what is genuinely still open. It exists to locate the

research gap for a master's thesis asking whether a graph neural network can learn physical laws from simulation data and generalise to unseen boundary conditions and unseen domains.

The survey covers 32 papers, all available in `thesis/literature/` and grouped one folder per section. They are grouped by what a method does, not by which architecture it uses, because the same architecture appears in several roles and the roles are what the argument turns on.

The recurring finding is that every family works inside the distribution it was trained on, and that they all stop at the same three obstacles: unseen boundary conditions, unseen geometries and, at the atomistic scale, unseen chemistries. Section 11 sets this out as a table, and the row that no paper fills, both obstacles tested together, is the thesis contribution.

3 How to read this survey

Seven families of methods are covered, one per section:

1. Physics-informed neural networks, which put the equation in the loss (Section 4).
2. Learned simulators on graphs, which learn the time-stepping map (Section 5).
3. Neural operators, which learn a whole family of problems at once (Section 6).
4. Physics built into the architecture, so that the law holds by construction (Section 7).
5. Hybrid methods, which keep the classical solver and only make it faster (Section 8).
6. Reinforcement learning, which decides where to spend compute rather than what the answer is (Section 9).
7. Atomistic and self-consistent-field methods, where the same question appears at the quantum scale (Section 10).

Each family gets three parts: the idea in a few sentences, what the family has solved with the papers that showed it, and what remains open. Paper summaries are deliberately short; the papers themselves are in `thesis/literature/` for deep reading.

Every displayed equation and every quoted number carries a pointer into its source, and the pointer names the paper as well as the location inside it, for example eq. (4) of Raissi et al., part I [1]. The intent is that any single statement here can be checked by opening one named paper at one named place.

4 Physics-informed neural networks

4.1 The idea

A physics-informed neural network (PINN) uses a network $u_\theta(t, x)$ to represent one solution of one problem, with θ the network weights, t time and x the space coordinate. The partial differential equation (PDE) itself becomes the loss. Write the equation as eq. (1) of Raissi, Perdikaris and Karniadakis, part I [1] does,

$$u_t + \mathcal{N}[u] = 0, \qquad x \in \Omega, \quad t \in [0, T], \tag{1}$$

where u_t is the time derivative of the solution, $\mathcal{N}[\cdot]$ is the nonlinear differential operator that encodes the physics, $\Omega \subset \mathbb{R}^D$ is the spatial domain in D dimensions and T is the final time.

The residual $f(t, x) := \partial_t u_\theta + \mathcal{N}[u_\theta]$, which is eq. (2) of the same paper [1, part I], is evaluated by automatic differentiation, meaning the same machinery that computes gradients for

training is reused to differentiate the network with respect to its input coordinates. Training then minimises the loss written as eq. (4) of Raissi et al., part I [1],

$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N_u}\sum_{i=1}^{N_u}\left|u_\theta(t_u^i, x_u^i) - u^i\right|^2}_{\text{initial and boundary conditions}} + \underbrace{\frac{1}{N_f}\sum_{i=1}^{N_f}\left|f(t_f^i, x_f^i)\right|^2}_{\text{the physics}}, \tag{2}$$

where $\{t_u^i, x_u^i, u^i\}$ are the N_u known values on the boundary and at the initial time, and $\{t_f^i, x_f^i\}$ are the N_f collocation points, which are locations inside the domain where the residual is checked. Nothing is solved to produce collocation points, so they carry no solution values.

4.2 Does this family need simulation data?

For a forward problem, no. The supervision is Equation (1) itself, plus the statement of the boundary and initial conditions, so a PINN can be trained with no solved instance of the problem at all. This is the property Raissi et al. emphasise, and they frame the residual term of Equation (2) as a regulariser that makes learning possible in the small-data regime [1, §1, §2.1].

Data enters in three other situations. Inverse problems use measurements, in practice often generated by a classical solver. Many later papers add labelled solution values to stabilise training, which makes them hybrids. Validation always needs a reference solution, but that is scoring rather than training.

The contrast with the rest of this survey is worth stating plainly, because it locates the thesis question. A PINN puts the physics in the loss, needs no data, and learns a single instance. The operator and simulator families of Sections 5 and 6 put the physics in the data and the architecture, need a solver to generate that data, and learn a whole family of problems. Transfer to unseen conditions is a question that can only be asked of the second kind.

4.3 What this family has solved

Raissi, Perdikaris and Karniadakis, part I [1]

Introduced the framework in two variants. The continuous-time model treats t as an ordinary network input [1, §2], while the discrete-time model places a multi-output network on the stages of a general Runge-Kutta scheme with q stages [1, §3, eqs. (7)–(11)], which lets implicit high-order time stepping and neural networks be used together. On Burgers' equation with viscosity $0.01/\pi$, trained on a few hundred boundary and initial points and optimised with L-BFGS, a quasi-Newton full-batch method, the reported relative L_2 error is 6.7×10^{-4}, about two orders of magnitude better than the Gaussian-process method the same three authors had published earlier, as reported in §2.1 and fig. 1 of part I [1]. A Schrödinger example uses 50 initial points, 50 periodic-boundary points and 20 000 collocation points placed by Latin hypercube sampling, with a network of five layers and one hundred neurons per layer [1, §2.2]. The authors state explicitly that the method is not a replacement for finite elements or spectral methods, which have fifty years of robustness behind them, and they leave uncertainty quantification open [1, §4].

Part II [2]

The inverse side, where the coefficients λ of the operator are unknown and are learned jointly with the solution from scattered space-time data, by adding them to the trainable parameters [2, eq. (5)]. On Burgers' equation, 2 000 points and a network of nine layers with twenty neurons per layer recover both coefficients to well under one percent on clean data, and the reported errors stay in the range of a few percent at one to ten percent

added noise [2, Tables 1–2]. On two-dimensional Navier-Stokes the identified equation is $u_t + 0.999(uu_x + vu_y) = -p_x + 0.01047\,(u_{xx} + u_{yy})$ against a true coefficient pair of 1 and 0.01, degrading only to 0.998 and 0.01057 at one percent noise, and the entire pressure field is recovered without a single pressure measurement, up to the additive constant that Navier-Stokes leaves undetermined [2, fig. 3]. A Korteweg-de Vries example recovers $u_t + 1.000\,uu_x + 0.0025002\,u_{xxx} = 0$ from two time snapshots [2, fig. 5]. This remains the setting where PINNs are genuinely strong, because sparse and noisy data enters the loss naturally, and it is also where simulation data does appear, since the training measurements are drawn from a high-resolution spectral solution.

Cuomo et al., survey [3]

An eighty-five page review organised around three questions: which network is used, how the physical knowledge is represented, and how it is integrated [3, §1.1]. It also fixes the vocabulary, noting that *physics-informed*, *physics-based*, *physics-guided* and *theory-guided* are used interchangeably in the literature. Useful as an index rather than a read-through.

4.4 What remains open

Krishnapriyan et al. [4]

The reference negative result, and the single most useful paper in this section. On one-dimensional convection with wave speed β, the plain PINN reaches a relative error of 7.5×10^{-1} at $\beta = 20$ and 9.6×10^{-1} at $\beta = 40$, that is nearly one hundred percent error after extensive hyperparameter tuning, on a problem with a closed form solution [4, §3.1, Table 1]. The same happens for reaction-diffusion as the diffusion coefficient ν grows [4, eq. (10), fig. 2]. Crucially they show this is not a lack of expressive power in the network; the soft PDE constraint of Equation (2) makes the optimisation landscape ill-conditioned, and lowering the weight on the residual term buys a smoother landscape at the price of a solution that no longer satisfies the physics [4, §4]. Two remedies help: curriculum regularisation, which trains at small β and reuses those weights to initialise the next β, cuts the $\beta = 20$ error to 9.8×10^{-3} [4, Table 1]; and sequence-to-sequence learning, which predicts one time step at a time, cuts a reaction-diffusion case from 5.1×10^{-1} to 1.2×10^{-2} [4, Table 2]. Both remedies require knowing in advance which parameter makes the problem hard, and the second quietly reinstates the time marching that PINNs were meant to avoid.

Wang, Yu and Perdikaris [5]

Supplies the mechanism. They derive the neural tangent kernel of a PINN, the kernel that describes how an infinitely wide network evolves under gradient descent, and prove it converges to a deterministic kernel that stays essentially constant during training [5, §4, Theorem 4.4]. The kernel splits into blocks for the boundary term and the residual term, and those blocks have very different eigenvalue scales, so the two parts of Equation (2) converge at very different rates [5, §7.1, figs. 2–3]. Combined with spectral bias, the tendency of such networks to fit smooth components long before sharp ones, this explains why high-frequency or multiscale solutions fail. The practical output is adaptive loss weighting.

Wang et al., expert's guide [6]

The state of the art assembled as a pipeline: non-dimensionalise the equation, add Fourier feature embeddings, apply random weight factorisation, use a modified multi-layer perceptron with two input encoders, then train with self-adaptive loss balancing, causal training, curriculum training and time marching [6, fig. 1, §6]. The composite

loss they minimise separates initial, boundary and residual terms explicitly [6, eqs. (2.5)–(2.8)]. The reported accuracies document a cliff, shown in Table 1: five decimal digits on smooth benchmarks, and roughly ten to twenty percent error as soon as the flow becomes multiscale or turbulent, despite multi-GPU training with 2×10^5 iterations per time window on the cylinder case [6, §7]. For the cylinder case no error is reported at all.

Zhang et al., limitations review [7]
Names three root causes [7, abstract]: the PDE loss approximates multiscale behaviour poorly and is ill-conditioned; convergence and error analysis are underdeveloped, so the mathematical guarantees that finite elements enjoy are missing; and the physics is integrated in a way that leaves the residual mismatched with the actual iteration error, meaning a small loss does not certify a small error.

Table 1: Best reported relative L_2 errors from the expert's guide pipeline [6, Table 1]. The gap between the top and bottom groups is three orders of magnitude, and it tracks how multiscale the solution is.

Benchmark	Relative L_2 error
Allen-Cahn equation	5.37×10^{-5}
Stokes flow	8.04×10^{-5}
Advection equation	6.88×10^{-4}
Lid-driven cavity, $Re = 3200$	1.58×10^{-1}
Kuramoto-Sivashinsky equation	1.61×10^{-1}
Navier-Stokes in a torus	2.45×10^{-1}
Navier-Stokes around a cylinder	not reported

Two conclusions carry into the rest of this survey. First, accuracy is not yet competitive with classical solvers on hard problems, and the original authors say so themselves. Second, and decisive for this project, a PINN is retrained from scratch for every new instance, so transfer across boundary conditions or domains is not merely unsolved, it is outside what the formulation can express.

5 Learned simulators on graphs

5.1 The idea

Represent the discretised system as a graph, with nodes carrying state and edges connecting neighbours, and learn the time-stepping map itself by message passing. Physics enters twice, through solver-generated training data and through the architecture's locality and permutation symmetry. A classical integrator usually remains, since the network predicts an acceleration or a rate and a plain Euler step turns that into the next state.

5.2 What this family has solved

Battaglia et al. [8]
A position paper rather than an experiment, and worth treating as such. It defines the graph network block as six functions, three that update edges, nodes and a global attribute and three that aggregate them, eq. (1) of Battaglia et al., and shows that interaction networks, message-passing networks, graph convolutions and self-attention are all special cases. The physics claims it makes are citations to earlier work, not results of its own, so this survey uses it for vocabulary and for the argument that locality and

permutation symmetry are the right inductive biases.

Sanchez-Gonzalez et al., graph network-based simulators, GNS [9]
Particle-based simulation of water, sand and deformable material in one framework. Particles are nodes, edges join particles within a fixed radius and are rebuilt every step as material moves, the network predicts per-particle acceleration, and a semi-implicit Euler integrator advances the state. Training is one step at a time, with random-walk noise added to input velocities so that rollouts stay stable, a detail without which the method does not work (§4.3 of Sanchez-Gonzalez et al.). Generalisation is the headline: a model trained on about 2 500 particles runs a scene that grows to 28 000 particles under continuous inflow, and on a domain 32 times larger it reaches 85 000 particles over 5 000 steps, eight times the training rollout length (§5.3).

Pfaff et al., MeshGraphNets [10]
The closest published template to this project. The graph carries two edge types: mesh edges fixed by the mesh connectivity, and world edges added dynamically between nodes that come close in space, which is what lets the model handle contact and self-collision, eq. (1) of Pfaff et al. Removing the world edges raises rollout error by 51 and 92 percent on two dynamic domains (§5), so the two-edge design is doing real work. Speed is the claim that matters commercially: 21 to 23 milliseconds per step against 820 for the ground-truth solver on cylinder flow, and 37 to 38 milliseconds against 11 015 on the aerofoil case, one to two orders of magnitude (Table 1 of Pfaff et al.). A model trained on 400-step trajectories rolls out stably for 40 000 steps (§5).

5.3 What remains open

MeshGraphNets is the one paper in this survey with genuine first-party evidence of mesh topology transfer. A model trained on rectangular cloth is tested on three disconnected fish-shaped flags and on a windsock with tassels, whose mesh averages 20 000 nodes against about 2 000 in training (§5 of Pfaff et al.). That is a different topology, not a scaled copy of the training domain, and any honest statement of the geometry gap has to concede it. The same paper also transfers to unseen physical parameters, testing aerofoil angles of attack from -35 to 35 degrees after training on -25 to 25, and Mach numbers from 0.7 to 0.9 after training on 0.2 to 0.7, with error rising from 11.5 to 12.4 and 13.1 respectively (§5).

What no paper in this family demonstrates is transfer to a different *type* of boundary condition. Node types such as walls, obstacles and inflows are fixed per domain, and none of the three papers holds out a condition type and tests it unseen.

Cost is the other honest caveat, and the GNS authors report it about their own method. Their timing appendix shows inference ranging from 51 percent to 345 percent of the ground-truth simulator's step time, meaning it is sometimes slower than the solver it imitates (appendix C.6 of Sanchez-Gonzalez et al.). The authors also state that mesh-based representations are future work, that no physical symmetries are imposed architecturally, and that efficiency gains are not yet realised (§6). Pfaff et al. report a decoherence effect in cloth after roughly 50 rollout steps, attributed to chaotic dynamics, and compensate by reporting a 50-step error alongside the full-trajectory one (appendix A.5.2). Both papers depend on a classical solver to generate thousands of training trajectories, which remains the dominant cost.

6 Neural operators

6.1 The idea

Instead of one solution, learn the operator: the map from a problem specification, meaning coefficients, forcing or initial data, to the solution function. One trained model then serves a whole family of instances, and the question of transfer becomes askable, because the model is supposed to handle inputs it has not seen.

6.2 What this family has solved

Lu, Jin and Karniadakis, DeepONet [11]

Gave the field its theoretical footing and a usable architecture. The input function is sampled at fixed sensor locations and fed to a branch network, the query location goes to a trunk network, and the output is their dot product, eq. (2) of Lu et al., a form justified by the operator universal-approximation statement of their eq. (1). Their Theorem 2 bounds the number of sensors needed for a target accuracy on an ordinary differential equation (ODE) operator, which is rare in this literature. On a diffusion-reaction problem the test error reaches about 10^{-5} from only 100 input-function samples (§4.3). Note a structural consequence of the design: the sensor locations must match between training and deployment, so this architecture cannot change resolution.

Li et al., graph kernel networks [12]

Made operator learning mesh-independent, and did it with message passing. The operator is built from a kernel integral, eq. (6) of Li et al., whose discretisation is exactly a message-passing update over neighbours within a radius, their eq. (10); the kernel is motivated by the Green's function of the elliptic problem, their eq. (5). Because the kernel is a function of coordinates rather than of grid indices, one parameter set serves many discretisations: trained at 16×16 and evaluated at 241×241 (fig. 1 and Table 1 of Li et al.), with error essentially flat from 85 to 421 points where a convolutional baseline degrades from 0.0253 to 0.1097 (Table 8).

Li et al., Fourier neural operator, FNO [13]

The standard baseline, obtained by restricting the kernel to a convolution and parameterising it in Fourier space, eqs. (3) and (4) of the FNO paper, keeping only the lowest twelve modes for the two-dimensional cases. It is the most accurate of the three on shared benchmarks, roughly 0.014 on Burgers across resolutions from 256 to 8 192 points (Table 3) and 0.0108 on Darcy flow against 0.0244 for the best classical reduced-basis competitor (Table 4). Two results give the family its reputation: genuine zero-shot super-resolution, training on $64 \times 64 \times 20$ data and evaluating at $256 \times 256 \times 80$ with no retraining (§5.4), and speed, 0.005 seconds per evaluation against 2.2 for the pseudospectral solver, which turns an 18-hour Bayesian inversion into two and a half minutes (§5.5). It should be said plainly that the paper contains no theorems; every claim is empirical.

Brandstetter, Worrall and Welling [14]

A fully neural message-passing solver, with no classical solver at inference. Boundary conditions and equation coefficients are supplied to the network as an explicit embedding θ_{PDE} inside the message and update functions, eqs. (8) and (9) of Brandstetter et al. The ablation on that embedding is the most useful number in the paper: on a wave problem mixing Dirichlet and Neumann conditions, error is 0.379 with the embedding and 17.591 without it, roughly forty-six times worse (Table 2). Telling the model what its boundary conditions are is therefore not optional. Accuracy beats both a fifth-order

finite-volume scheme and two FNO variants on the harder generalisation setting, 3.70 against 15.94 and 5.98 (Table 1), at 0.08 seconds per rollout against 1.7 for the classical scheme. The paper also contributes the stability techniques, temporal bundling and the pushforward trick, that tame autoregressive error.

6.3 What remains open: what this project targets

Before the two gap papers, the position of the three foundations should be stated precisely, because it is easy to overstate. None of DeepONet, the graph kernel network or FNO varies the domain shape in any experiment: the domains are an interval, a unit square and a torus. None tests a single trained model across boundary-condition types, and none tests transfer to a different governing equation. Only DeepONet reports an out-of-distribution input experiment at all (fig. 5, §4.1.2 of Lu et al.). FNO argues in a short paragraph that its architecture is not restricted to periodic conditions, but supports this by using different fixed conditions in separate experiments rather than by testing one model across types (§5 of the FNO paper). What this family solved is resolution and same-distribution inputs, which is a genuine achievement and a narrower one than the phrase "learning the operator" suggests.

Mousavi, Mishra and De Lorenzis [17]

States the boundary-condition gap directly, ICML 2026: the conditioning of operators on boundary data "has received very little attention", and existing methods are restricted to Cartesian geometries, fail to support complex conditions, or degrade when conditions vary across samples (§1 of Mousavi et al.). Their remedy is an extender module that maps boundary data of any type, Dirichlet, Neumann, Robin or mixed, into a latent function defined over the whole domain, so that any domain-to-domain operator can consume it, eqs. (4) to (6). Results are strong: 0.02 to 0.03 percent error on Poisson against 2.23 to 3.61 for the previous boundary-specialised baseline (fig. 2a), and elasticity error cut from 30.6 to 11.0 percent (fig. 2b). Against a finite element reference the learned model is comparable in accuracy at 13.8 against 12.2 percent and roughly twenty thousand times faster, 5.9 milliseconds against 125 seconds (Table 1). Their main backbone, RIGNO, is a message-passing graph network.

Huang et al., Schwarz neural inference [18]

States the geometry gap directly, ICLR 2026: existing operators "lack the ability to generalize to entirely novel geometries that differ significantly from those present in the training data distribution", and the need to generate new data for each new shape is what blocks industrial use (§1 of Huang et al.). Their remedy is local-to-global: train the operator only on small simple polygons, then at inference partition an arbitrary domain with a classical graph partitioner and stitch subdomain solutions with an additive Schwarz iteration, eqs. (2) and (3). This is genuine zero-shot geometry transfer, including domains with holes, that is a change of topology, and a dolphin-shaped domain at 4.5 percent error. Errors fall from 22–28 percent to about 2.1 percent on Laplace problems and from 16–167 percent to 5–8 percent on Darcy (Table 1). One detail matters for this project: their learned operator is a transformer, not a graph network, and their own appendix reports that MeshGraphNets generalises better across domains than the transformer they adopted, with replacing the transformer by a graph network named as future work (appendix H.4 and appendix G).

Now the gap. Three papers sit closest to it: Mousavi et al. and Huang et al. just described, and Brandstetter et al. from the previous subsection. Two things can vary between training and deployment, the type of boundary condition and the shape of the domain. Each of the three papers handles one of them and holds the other fixed, and none handles both at once.

Mousavi et al. vary boundary conditions richly but fix one geometry per dataset, and every cross-geometry or cross-condition-type result in that paper comes from fine-tuning on between 8 and 512 samples of the target setting, not from zero-shot evaluation (fig. 5 and appendix F.4). Huang et al. achieve zero-shot transfer to new shapes but never withhold a boundary-condition type; only condition values vary at test time, and the types are all present in training (appendix H.1). Brandstetter et al. use a graph network and handle mixed condition types, but train on a mixture of them rather than holding one out, and never vary the domain shape at all, so the geometry language in that paper describes sampling regularity rather than topology.

The untested combination is therefore specific: unseen geometry and unseen boundary-condition type, varied together in one experiment, on a mesh-based graph network, evaluated with a solver in the loop so that correctness is guaranteed rather than hoped for. Nothing in this collection does that, which is the open row of Table 2.

Two further observations support attacking it with a graph network rather than a transformer or a Fourier operator. Structure helps out of distribution: Shaffer et al. report 2.14 percent against 8.90 percent on unseen geometries for an architecture-matched ablation (Section 7). And Huang et al. measured graph networks generalising across domains better than the transformer they themselves adopted. Neither observation is proof, and both are the kind of evidence a proposal should cite rather than assert.

7 Physics built into the architecture

7.1 The idea

Rather than penalising physics violations in the loss, make them impossible: choose an architecture whose outputs satisfy the law by construction. The loss then only has to fit data, because the physics is already true of every function the network can represent.

7.2 What this family has solved

Greydanus, Dzamba and Yosinski, Hamiltonian networks [15]

Learn a scalar $H_\theta(q, p)$ and obtain the dynamics as its symplectic gradient, following Hamilton's equations, eq. (2) of Greydanus et al., with the network trained on the residual of those equations, their eq. (3). Energy is then conserved by construction rather than approximately. The effect is dramatic on the metric that matters: on the mass-spring system the energy error falls from 170 to 0.38 in the paper's units, and on a pendulum learned from pixels from 9.3 to 0.15, against an unconstrained baseline of the same size (Table 1 of Greydanus et al.).

Cranmer et al., Lagrangian networks [16]

The same trick without requiring canonical coordinates: learn a Lagrangian and recover accelerations by solving the Euler-Lagrange equation through the network, eq. (6) of Cranmer et al. This widens the reachable systems, and the paper's showcase is a relativistic particle in non-canonical coordinates where the Hamiltonian version fails. On a double pendulum the energy discrepancy is 0.4 percent of maximum potential energy against 8 percent for the unconstrained baseline (§5 of Cranmer et al.). The cost is a Hessian inverse scaling as $O(d^3)$ in the number of coordinates.

Shaffer et al., structure-preserving neural PDEs [19]

The paper that carries this section, because it is the only one of the three about PDEs on meshes. It learns a reduced finite-element basis conditioned on the mesh together with a discrete flux, then solves the resulting nonlinear system at inference, eqs. (3),

(7) and (9) of Shaffer et al. Finite element exterior calculus makes conservation structure and Dirichlet conditions exact by construction, and Table 3 of that paper reports boundary error of exactly zero against 10^{-3} for the unconstrained encoder. On standard benchmarks it improves on the best prior learned method by 29 percent for elasticity and 71 percent for pipe flow (Table 2, §4.1).

7.3 What remains open

One caveat about how this survey uses the Hamiltonian and Lagrangian papers. They are small conservative systems of ordinary differential equations, a pendulum, a double pendulum, a two-body problem, and the authors of the Hamiltonian paper state the restriction themselves: the method assumes a conserved quantity exists and cannot account for friction (§3.2 of Greydanus et al.). There are no spatial domains, meshes or boundary conditions in that setting, so those papers say nothing directly about the transfer axes this survey cares about. They establish the principle; they do not establish it for PDEs.

A second caveat concerns the strength of the claim. All three papers compare a structurally constrained architecture against an unconstrained one, not against a soft-constrained one, so the defensible statement is that built-in structure beats no structure. Whether it beats a penalty in the loss is untested here, and stating otherwise would overreach. The comparison is cleanest in Shaffer et al., whose baseline shares the same encoder and differs only by the removal of the structural constraint, which is a real ablation rather than a different model.

What that ablation shows is the reason this family appears in the survey at all. On unseen geometries the structured model reaches 2.14 percent error against 8.90 percent for the architecture-matched baseline, and on the harder decomposition benchmark the coordinate-based baselines fail outright while the structured model still functions (Table 1 of Shaffer et al.). The gain is largest exactly where this project is aimed, at out-of-distribution geometry.

The remaining gaps are stated by Shaffer et al. in their limitations section: two dimensions only, steady state only, a computational mesh and specified boundary conditions assumed available, extra solve cost against direct regression, and a learned constitutive model class restricted by stability requirements. Their boundary-condition transfer evidence is also weak, being a single qualitative experiment that swaps inlet and outlet labels without retraining and reports no error figure, while the main out-of-distribution benchmarks explicitly hold boundary conditions fixed as geometry varies. Extending exact structure preservation to time-dependent, dissipative and coupled multiphysics problems is open, and this is where the project's structured-versus-unstructured ablation sits.

8 Hybrid methods: accelerate the classical solver

8.1 The idea

Keep the classical solver as the authority and use learning only to reduce its iteration count, through warm starts, learned preconditioners or learned solver parameters. Correctness is inherited from the solver, because the stopping criterion is unchanged, so only speed is at stake. This is the mode in which a wrong prediction costs time rather than trust.

8.2 What this family has solved

Eshaghi et al., neural operator warm starts, NOWS [20]

The cleanest statement of the warm-start idea. A neural operator trained offline maps the problem specification to an approximate solution, and that prediction is used only as the initial guess $u_0 = \mathcal{G}_\theta(a)$ of a Krylov solver, eq. (5) of Eshaghi et al., which then runs to its usual tolerance under the update rule of their eq. (6). Discretisation,

preconditioner and stopping criterion are untouched. Reported iteration reductions of two- to tenfold and runtime savings of 25 to 90 percent (abstract and §1 of Eshaghi et al.); about 90 percent fewer iterations on an isogeometric plate with voids across 50 unseen void configurations (§3.4, fig. 4c); a Burgers case falling from 9.62 to 4.75 seconds on average (§3.5, fig. 5b); and a smoke-plume Navier-Stokes case falling from 69 703 to 17 438 total iterations, and from 294.74 to 93.85 seconds, over 150 runs (§3.6). Network inference time is included in the reported totals, though the offline data generation and training cost is not.

Zhang, Li and Saad, graph neural multilevel preconditioners [21]
The benchmarking standard this project must meet, and a genuine graph network: message passing within each level of a classical algebraic multigrid hierarchy, with interpolation and restriction replaced by learned bipartite cross-attention, so that restriction need not be the transpose of interpolation, eqs. (10) and (17) of Zhang et al. Evaluated on 867 non-symmetric SuiteSparse matrices from one thousand to one hundred thousand unknowns, against algebraic multigrid, incomplete factorisation, Jacobi and a single-level graph preconditioner, all under one identical solver protocol (§4.1.1). The learned preconditioner wins on iteration count in 62.1 percent of cases against the single-level version, rising as the tolerance tightens (Table 1), and the multilevel form reaches 72 percent (Table 2). Failure rates are far below incomplete factorisation, which fails to construct on 40.14 percent of these matrices (Table 3). Setup cost is disclosed rather than hidden: 210.90 seconds on average against 103.22 for algebraic multigrid, with the solve phase five times faster (Table 4).

Arisaka and Li, non-intrusive meta-solving [22]
Learning wrapped around a legacy solver that cannot be differentiated at all, which is the industrial case. A network predicts solver inputs, and since the solver is a black box, its gradient is estimated by a forward probe whose variance is cut by a control variate built from a differentiable surrogate, eq. (4) with the variance result of their eq. (7). Demonstrated through PETSc on Poisson, biharmonic and three-dimensional elasticity problems: average iterations fall from 73.74 to 31.54 on the biharmonic case and from 88.71 to 25.61 on elasticity (Table 3 of Arisaka and Li), and by 74 percent against a supervised-learning baseline for Jacobi (Table 2 and §5.1).

Zou, Xu and Zhang, survey [23]
Organises the design space into method selection, component optimisation and parameter tuning. Two of its conclusions are useful here: the field has no standard benchmark datasets, which makes published comparisons hard to trust (§5.2 and §5.4 of Zou et al.), and graph networks are named as an underexplored direction precisely because a sparse matrix already is a graph (§5.4).

8.3 What remains open

Wu et al., are hybrid solvers reliable? [24]
The cautionary paper this project has to answer. It studies hybrids that alternate a classical smoother with a neural correction, and identifies a central failure mode named false fixed points: the iteration update shrinks to around 10^{-4} while the true residual sits at about 10^{1}, so the solver looks converged and is not (fig. 1 and §5.2 of Wu et al.). It also prices the fashionable remedy, since training through unrolled iterations costs 10.90 times the training time and 1.61 times the memory for a convergence gain of only 1.16 to 1.43 times in one case, and 5.96 times the time with 8.53 times the memory in another (Table 1, §5.4). Their own fix, using the physical residual rather than the fixed-

point update inside Anderson acceleration, converges to 10^{-10} on a Helmholtz problem where the standard version stalls near 10^{-2} (§5.5, fig. 4a), with wall-clock gains up to 4.50 times (Table 2). The honest conclusion they draw is that no training objective is universally right and that unrolled training should be used with caution (§6).

Reading the five papers of this section together produces the sharpest gap statement in this survey, and it is a gap of omission rather than of failure.

Not one of the five tests transfer to a different *type* of boundary condition. NOWS and the reliability study both keep homogeneous Dirichlet conditions throughout and vary only source terms, coefficient fields or resolution. The preconditioner paper operates on bare sparse matrices, where boundary conditions have already been compiled away and no longer exist as a concept. The meta-solving paper holds the unit domain fixed in every experiment.

Geometry fares only slightly better. NOWS varies void configurations in one example (§3.4), and the preconditioner paper generalises across unseen grid sizes for one equation family, where a single trained model beats per-matrix-trained baselines at every size (Table 8, appendix A.4). Nothing here approaches transfer to a genuinely new domain shape.

Two of the papers name this project's exact next step as their own future work. Eshaghi et al. propose extending warm starts to graph-based operator architectures for unstructured meshes (§4), and Zou et al. argue graph networks are the natural fit for solver components (§5.4). The combination of warm starts, graph networks on unstructured meshes, and transfer across boundary conditions and geometries is asked for in this literature and not answered by it.

9 Reinforcement learning for solver control

9.1 The idea

Reinforcement learning decides where to spend compute rather than what the solution is. Adaptive mesh refinement is the mature case: an agent looks at the current mesh, chooses which elements to subdivide, and is rewarded for error removed per element added. The physics still comes from a conventional finite element solver, which is called after every refinement step.

9.2 What this family has solved

Yang et al. [25]

Formulates refinement as a Markov decision process over the whole mesh, with reward equal to the normalised error reduction from one refinement step, eq. (1) of Yang et al., and the discounted return of eq. (2). Three interchangeable policy heads are compared, a per-element network, a hypernetwork and a graph network built from interaction networks, and the graph network is the strongest on advection problems, that is on moving features (§4.3 and §6.1 of Yang et al.). Reported to match or beat the Zienkiewicz-Zhu error estimator on 20 of 24 cases (§6 of Yang et al.), and a policy trained on an 8×8 mesh deploys on 200×200, which is 625 times more elements (§6.3, fig. 7 of Yang et al.).

Foucart, Charous and Lermusiaux [26]

Recasts the same task locally: the agent sees one element at a time and chooses to coarsen, do nothing or refine, with a reward that trades accuracy against resource cost through a tunable coefficient, eq. (4) of Foucart et al. Because the observation is purely local, the policy is independent of mesh size by construction. Reported to preserve

solution features at under a quarter of the cost of a classical heuristic in unsteady one-dimensional advection (§4.3 of Foucart et al.), and to need roughly a tenth of the elements on a two-dimensional advecting ring (§4.7 of Foucart et al.). Notably, §4.2 of the same paper transfers one trained policy to a different domain, different boundary conditions and a different forcing function, which is a genuine unseen-condition test. The policy is a small multilayer perceptron, not a graph network.

Freymuth et al., swarm refinement [27]

The most developed of the three, and the only one whose policy is a message-passing graph network over the mesh. Every element is an agent with a binary refine decision, agents split into child agents, and a responsibility mapping propagates credit as elements subdivide, eqs. (2), (4) and (6) of Freymuth et al. Evaluated on seven problem families including Stokes flow, linear elasticity and three-dimensional Poisson, against uniform refinement, the Zienkiewicz-Zhu estimator, two oracle heuristics with privileged access to a fine reference, and reimplementations of both papers above. A policy trained on unit-square domains deploys on a 20×20 spiral domain, meshing it in 12.2 seconds against about twenty minutes for the uniform reference, roughly a hundredfold speedup at a mesh error below 10^{-3} (§4.5.1, figs. 15 and 16 of Freymuth et al.). As few as ten training problems suffice, with gains up to a hundred (§4.3.4 of the same paper).

9.3 What remains open

Freymuth et al. state the binding cost in §5 of their paper: the finite element system must be solved after every refinement step, both in training and at deployment, which becomes expensive on fine meshes, and the reward needs an expensive reference solution during training, so the achievable resolution is capped by that reference. Their own proposed remedy, learning to predict refinement regions directly from the problem conditions without repeated solves, is left as future work. The method is also refinement-only, stationary, and demonstrated on linear problems.

The other two papers leave related gaps. Yang et al. refine one element per step, exclude coarsening because it needs a two-objective treatment of error against cost, and describe their explanation for beating greedy baselines as a conjecture rather than a result (§7 of Yang et al.). Foucart et al. flag that changing the accuracy-versus-cost coefficient requires retraining, with their tunable-policy workaround described as ongoing research.

Two conclusions matter here. First, transfer is taken seriously in this family, and Freymuth et al. do test unseen domain sizes and unseen material parameters, although the geometry result relies on a data-augmentation trick that mimics being a patch of a larger mesh rather than on bare geometric generalisation. Second, and decisively, in none of the three papers does the network ever predict a solution field. The graph network chooses where to refine and a classical solver supplies the physics, so this family is a control layer over solvers and not a route to learning the physics itself.

10 The same gap at the atomistic scale

10.1 Why this section exists

Machine-learned interatomic potentials (MLIPs) replace density functional theory (DFT) inside molecular dynamics, and that community has arrived independently at the same obstacle as the PDE community. The parallel is close enough to be useful: chemistry is to a potential what boundary conditions and geometry are to an operator, namely the thing the model was never shown and is expected to handle anyway.

10.2 The obstacle, stated by the field itself

Creed et al., six open questions [28]

A 23-author discussion paper, and the strongest single source for the parallel just drawn. It organises the state of universal potentials around six questions printed as its section headings: what the minimal definition of an atomistic foundation model is (§1), whether the field needs more data, better data or better models (§2), whether long-range interactions are handled and whether that matters (§3), whether such models can discover genuinely new physics (§4), whether they scale to more useful simulations (§5), and how anyone knows whether a potential is any good (§6). The verdict in §4 is that models learn well within known chemistry and physics but struggle outside their training distribution, and the amplification is brutal: energy errors of 0.008 eV can produce migration-barrier errors of 0.74 eV, two orders of magnitude larger (§4.1 of Creed et al.). The paper offers no resolution by design.

Wong and Yang, bias and fine-tuning [29]

An unseen-chemistry study by construction: a universal MACE potential is applied to a liquid electrolyte whose components are absent from its training set. The pretrained model reaches 21.24 meV per atom on an independent test set, and iterative fine-tuning brings this to 5.79, while the naive strategy of sampling fine-tuning data with the biased model itself plateaus near 10 and produces unphysical dynamics including spurious deprotonation (Tables 1 and 2, fig. 5 of Wong and Yang). The lesson transfers directly to this project: data generated by a biased model cannot cure that model's bias. Only one architecture was tested, which the authors state as a limitation.

Chiang et al., MLIP Arena [30]

A benchmark platform, NeurIPS 2025 datasets and benchmarks track, that evaluates 13 pretrained potentials on physics-grounded tests rather than energy regression alone: smoothness of the potential energy surface, stability under extreme conditions, and robustness under distribution shift measured by a differential-entropy notion of how out-of-distribution a structure is (§2.3 of Chiang et al.). Architecturally equivariant models keep rotational equivariance exactly, while non-equivariant ones degrade as distributional surprise grows, which is direct evidence that built-in structure buys robustness. Their related-work section states the field-wide position plainly: many models saturate existing metrics yet fail to extrapolate to unseen chemistry, strained configurations or finite-temperature behaviour.

10.3 The solver side: learned initial guesses

Eberhard et al., solver-aligned initialisation learning, SAIL [31]

The atomistic twin of the warm-start idea of Section 8, and an equivariant graph network. Instead of fitting a converged ground state, it learns the starting point of the self-consistent field cycle by differentiating through the solver itself, so the objective is alignment with the solver's trajectory rather than similarity to a target, eqs. (2) and (5) of Eberhard et al. The cycle still runs to convergence, so correctness is inherited exactly as in the hybrid family. Reported iteration-count reductions of 37, 33 and 28 percent for three functionals on molecules up to four times larger than the training set, and a 1.35-fold wall-clock speedup on molecules ten times larger (Table 1 and figs. 1 and 5). One measurement is worth remembering for cost modelling: building the Fock matrix accounts for 97.8 percent of the work in an iteration (fig. 2 of the same paper).

Hazra, Patil and Sanvito [32]

Included as the field's earlier starting point rather than as evidence of transfer. A

small dense network, not a graph network and not equivariant, predicts the one-particle density matrix from atomic coordinates, cutting the cycle to one or two iterations and giving a three- to fivefold speedup (Table I and §III.A of Hazra et al.). Equivariance is imposed by hand, by fixing molecular orientation, which the authors describe as not general. Decisively for this survey, one network is trained per molecule on configurations of that same molecule, and no cross-molecule test is reported, so the paper makes no transfer claim at all.

10.4 What remains open

Eberhard et al. list their own boundaries in their limitations section: a separate model is needed per exchange-correlation functional and, for matrix representations, per basis set; the evaluation covers only hydrogen, carbon, nitrogen, oxygen and fluorine; and transfer to heavier elements, to non-equilibrium geometries and to periodic systems is untested, with the last described as remaining open. So even the best transferable-initialisation result is transfer in system size, not in chemistry.

That distinction matters for how this survey uses the analogy. Unseen size is demonstrated, unseen chemistry is not, which is precisely the pattern found in Section 6, where resolution transfer is solved and boundary conditions and geometry are not. The two scales are not merely similar in spirit, they are stuck on the same step.

11 Synthesis: the generalisation matrix

Three observations close the survey.

Every family performs inside its training distribution, and every family stops in the same place: at the things real applications vary, since every patient, reservoir and molecule is a new boundary condition, geometry or chemistry.

The most promising lever reported so far is structure in the architecture rather than volume of data [15, 19], which is a testable hypothesis rather than a folklore claim.

And the hybrid deployment mode [20, 21] changes what failure costs: outside the distribution a surrogate is silently wrong, while a warm start merely saves fewer iterations. Measuring where those two behaviours separate is precisely the proposal built on this survey.

References

[1] M. Raissi, P. Perdikaris and G. E. Karniadakis. Physics informed deep learning (part I): data-driven solutions of nonlinear partial differential equations. arXiv:1711.10561, 2017.

[2] M. Raissi, P. Perdikaris and G. E. Karniadakis. Physics informed deep learning (part II): data-driven discovery of nonlinear partial differential equations. arXiv:1711.10566, 2017.

[3] S. Cuomo et al. Scientific machine learning through physics-informed neural networks: where we are and what's next. arXiv:2201.05624, 2022.

[4] A. S. Krishnapriyan et al. Characterizing possible failure modes in physics-informed neural networks. NeurIPS 2021; arXiv:2109.01050.

[5] S. Wang, X. Yu and P. Perdikaris. When and why PINNs fail to train: a neural tangent kernel perspective. arXiv:2007.14527, 2020.

[6] S. Wang et al. An expert's guide to training physics-informed neural networks. arXiv:2308.08468, 2023.

[7] W. Zhang et al. Physics-informed neural networks as intelligent computing technique for solving PDEs: limitation and future prospects. arXiv:2411.18240, 2024.

Generalisation axis	Status	Evidence
New forcing or coefficients, within the training distribution	largely solved	operator learning [11, 13]
New mesh resolution	solved for spectral operators	zero-shot super-resolution [13]; mesh-invariant kernels [12]
Larger systems, longer rollouts	partly solved	particle and mesh simulators [9, 10], with stability tricks [14]
Unseen boundary-condition types	**open**	gap stated in print, and closed only by fine-tuning on the target setting [17]; mixed types trained as a mixture, never held out [14]
Unseen geometries	**partly open**	mesh topology transfer demonstrated for cloth [10]; zero-shot for shapes with holes, but with a transformer, not a graph network [18]; structure helps out of distribution [19]
Both together, in one experiment	**open**	each of the three closest papers handles one and fixes the other [14, 17, 18]
Unseen chemistries (atomistic)	**open**	field-stated open questions [28]; bias on unseen chemistry [29]; benchmark evidence [30]
A different governing equation	far open	no credible general result to date

Table 2: What is solved and what is open, per generalisation axis. The bold rows are the research gap this project addresses; the last row is deliberately out of scope.

[8] P. W. Battaglia et al. Relational inductive biases, deep learning, and graph networks. arXiv:1806.01261v3, October 2018. Position paper; no first-party experiments.

[9] A. Sanchez-Gonzalez et al. Learning to simulate complex physics with graph networks. ICML 2020, PMLR vol. 119; arXiv:2002.09405.

[10] T. Pfaff et al. Learning mesh-based simulation with graph networks. ICLR 2021; arXiv:2010.03409v4.

[11] L. Lu, P. Jin and G. E. Karniadakis. DeepONet: learning nonlinear operators based on the universal approximation theorem of operators. arXiv:1910.03193v3, April 2020.

[12] Z. Li et al. Neural operator: graph kernel network for partial differential equations. arXiv:2003.03485, March 2020.

[13] Z. Li et al. Fourier neural operator for parametric partial differential equations. ICLR 2021; arXiv:2010.08895v3.

[14] J. Brandstetter, D. E. Worrall and M. Welling. Message passing neural PDE solvers. ICLR 2022; arXiv:2202.03376v3.

[15] S. Greydanus, M. Dzamba and J. Yosinski. Hamiltonian neural networks. NeurIPS 2019; arXiv:1906.01563v3.

[16] M. Cranmer et al. Lagrangian neural networks. arXiv:2003.04630v2, July 2020.

[17] S. Mousavi, S. Mishra and L. De Lorenzis. Imposing boundary conditions on neural operators via learned function extensions. ICML 2026, PMLR vol. 306; arXiv:2602.04923v2, May 2026.

[18] J. Huang et al. Operator learning with domain decomposition for geometry generalization in PDE solving. ICLR 2026; arXiv:2504.00510v2, February 2026.

[19] B. D. Shaffer, S. Koohy, B. Kinch, M. A. Hsieh and N. Trask. Structure-preserving learning improves geometry generalization in neural PDEs. ICML 2026, PMLR vol. 306; arXiv:2602.02788v2, June 2026.

[20] M. S. Eshaghi, C. Anitescu, N. Valizadeh et al. NOWS: neural operator warm starts for accelerating iterative solvers. arXiv:2511.02481v4, May 2026.

[21] Z. Zhang, R. P. Li and Y. Saad. Graph neural multilevel preconditioners for iterative solvers. KDD '26; arXiv:2607.28456, July 2026.

[22] S. Arisaka and Q. Li. Accelerating legacy numerical solvers by non-intrusive gradient-based meta-solving. ICML 2024, PMLR vol. 235; arXiv:2405.02952.

[23] H. Zou, X. Xu and C.-S. Zhang. A survey on intelligent iterative methods for solving sparse linear algebraic equations. arXiv:2310.06630, 2023.

[24] Y. Wu, J. W. van Beek, V. Dolean and A. Heinlein. Are deep learning based hybrid PDE solvers reliable? Why training paradigms and update strategies matter. arXiv:2602.06842v2, June 2026.

[25] J. Yang, T. Dzanic, B. Petersen et al. Reinforcement learning for adaptive mesh refinement. AISTATS 2023, PMLR vol. 206; arXiv:2103.01342.

[26] C. Foucart, A. Charous and P. F. J. Lermusiaux. Deep reinforcement learning for adaptive mesh refinement. arXiv:2209.12351, 2022.

[27] N. Freymuth, P. Dahlinger, T. Würth, S. Reisch, L. Kärger and G. Neumann. Adaptive swarm mesh refinement using deep reinforcement learning with local rewards. arXiv:2406.08440v2, January 2026; extended version of the earlier ASMR conference paper, method named ASMR++.

[28] I. Creed, T. Rein, I. Vitenburgs et al. (23 authors). Six open questions in machine-learned interatomic potential foundation models. arXiv:2606.07327v2, June 2026.

[29] N. H. Wong and J. H. Yang. Bias in universal machine-learned interatomic potentials and its effects on fine-tuning. arXiv:2603.10159v2, June 2026.

[30] Y. Chiang, T. Kreiman, C. Zhang et al. MLIP Arena: advancing fairness and transparency in machine learning interatomic potentials via an open, accessible benchmark platform. NeurIPS 2025, datasets and benchmarks track; arXiv:2509.20630v2.

[31] E. S. Eberhard, V. Kotsev, T. Güthle and S. Günnemann. Transferable SCF-acceleration through solver-aligned initialization learning. arXiv:2604.21657v2, May 2026.

[32] S. Hazra, U. Patil and S. Sanvito. Predicting the one-particle density matrix with machine learning. arXiv:2401.06533, 2024.